



Arquivos em Python

:::: Introdução a Algoritmos



Agora, é sua vez!

Um sistema precisa armazenar 5 números inteiros digitados por um usuário.

Depois de armazenar todos os valores, o sistema deve exibir esses números na ordem inversa da leitura.

Escreva um programa que:

- leia os 5 números
- armazene os valores em uma lista
- exiba os valores na ordem inversa

Pensando no Problema

Entrada:

- 5 números inteiros

Processamento:

- armazenar os valores em uma lista
- percorrer a lista do final para o início
- acessar os elementos na ordem inversa

Saída:

- valores exibidos na ordem inversa

Casos de Teste

Entrada	Saída Esperada
1, 2, 3, 4, 5	5, 4, 3, 2, 1
10, 20, 30, 40, 50	50, 40, 30, 20, 10
7, 3, 9, 1, 8	8, 1, 9, 3, 7

Implementação

```
1 valores = []
2
3 for i in range (5):
4     numero = int(input())
5     valores.append(numero)
6
7 i = len(valores) - 1
8
9 while i >= 0:
10     print(valores[i])
11     i -= 1
```

Problema

Finalizei o dia de trabalho, desliguei o computador. Preciso continuar o trabalho no dia seguinte, com a mesma lista, a partir de onde eu parei.

- Solicito ao usuário outra vez? (*O usuário vai estar disponível? Ele vai fornecer os mesmos dados?*)
- Preciso que os dados sejam salvos e persistam após o computador ser desligado.

Solução: arquivos

Um arquivo é uma coleção de dados persistentes armazenada em um dispositivo de armazenamento secundário (por exemplo o HD).

- Diferente das variáveis, listas e dicionários, os arquivos mantêm os dados salvos "permanentemente" para uso futuro
- Para o Python, um arquivo é visto como uma sequência de bytes
- O S.O. gerencia onde esses bytes estão fisicamente, e o Python nos dá ferramentas para ler ou escrever nessa sequência

O Ciclo de Vida

Para manipular qualquer arquivo no seu código, o Python sempre segue uma regra de três passos:

- Abrir (open): Cria uma ponte entre o seu script Python e o arquivo no sistema operacional.
- Processar (Ler/Escrever): Transfere os dados através dessa ponte.
- Fechar (close): Corta a ponte, salvando as alterações e liberando a memória do sistema operacional.

O Ciclo de Vida

```
# Abrindo no modo de escrita ('w')
arquivo = open("exemplo.txt", "w", encoding="utf-8")

# Escrevendo algo
arquivo.write("Olá, Mundo!")

# FECHAR É OBRIGATÓRIO AQUI!
arquivo.close()
```



Agora, é sua vez!

Um sistema precisa armazenar 5 números inteiros digitados por um usuário.

Depois de armazenar todos os valores, o sistema deve exibir esses números na ordem inversa da leitura.

Escreva um programa que:

- leia os 5 números
- armazene os valores em uma lista
- exiba os valores na ordem inversa
- armazene a lista invertida em um arquivo com extensão “.txt”

Implementação

```
1 valores = []
2
3 for i in range(5):
4     numero = int(input())
5     valores.append(numero)
6
7 i = len(valores) - 1
8
9 arquivo = open("arquivo.txt", "w")
10
11 while i >= 0:
12     print(valores[i])
13
14     arquivo.write(str(valores[i]) + "\n")
15
16     i -= 1
17
18 arquivo.close()
```

Converte para string antes de escrever no arquivo

O Ciclo de Vida

Nome do arquivo, pode acompanhar o caminho. Por exemplo:
"/home/usuario/documentos/exemplo.txt"

```
# Abrindo no modo de escrita ('w')
arquivo = open("exemplo.txt", "w", encoding="utf-8")

# Escrevendo algo
arquivo.write("Olá, Mundo!")

# FECHAR É OBRIGATÓRIO AQUI!
arquivo.close()
```

O Ciclo de Vida

O modo de abertura. Pode ser aberto para leitura ou escrita.

```
# Abrindo no modo de escrita ('w')
arquivo = open("exemplo.txt", "w", encoding="utf-8")

# Escrevendo algo
arquivo.write("Olá, Mundo!")

# FECHAR É OBRIGATÓRIO AQUI!
arquivo.close()
```

O Ciclo de Vida

```
# Abrindo no modo de escrita ('w')
arquivo = open("exemplo.txt", "w", encoding="utf-8")

# Escrevendo algo
arquivo.write("Olá, Mundo!")

# FECHAR É OBRIGATÓRIO AQUI!
arquivo.close()
```

Parâmetro opcional, mas altamente recomendada a utilização.

Abrindo arquivos

A Função `open()` e seus Modos

- `'r'` (*Read*): Modo de leitura. O arquivo precisa já existir, ou o Python gera um erro (*FileNotFoundError*).
- `'w'` (*Write*): Modo de escrita. Se o arquivo já existir, ele apaga todo o conteúdo e começa do zero. Se não existir, ele cria um arquivo novo.
- `'a'` (*Append*): Modo de adição. Ele abre o arquivo e posiciona o "cursor" no final. Adiciona-se ao conteúdo existente, sem apagar nada.

Fechando arquivos

A Função `close()` é de uso obrigatório

- No modo escrita, ao não fechar um arquivo a escrita não acontece no disco. Perde-se dados.
- Adicionalmente, ele fica bloqueado para outros acessos.
- No modo leitura, não fechar um arquivo consome uma unidade do limite máximo de arquivos que um único programa pode manter aberto (por exemplo, 1024 no linux).

Boa prática: *with*

Gerenciador de contexto *with*

- Se ocorrer algum erro antes da chamada à função `close()`, os problemas do não fechamento podem ocorrer
- *with* garante que o arquivo será fechado automaticamente assim que o bloco de código terminar, mesmo que ocorra um erro no meio do caminho.

Exemplo

```
1 valores = []
2
3 for i in range(5):
4     numero = int(input())
5     valores.append(numero)
6
7 i = len(valores) - 1
8
9 with open("arquivo.txt", "w") as arquivo:
10     while i >= 0:
11         print(valores[i])
12
13         arquivo.write(str(valores[i]) + "\n")
14
15         i -= 1
```

Exemplo

```
1 valores = []
2
3 for i in range(5):
4     numero = int(input())
5     valores.append(numero)
6
7 i = len(valores) - 1
8
9 with open("arquivo.txt", "w") as arquivo:
10     while i >= 0:
11         print(valores[i])
12
13         arquivo.write(str(valores[i]) + "\n")
14
15         i -= 1
```

O python garante o fechamento do arquivo neste ponto.



Agora, é sua vez!

Uma empresa de logística precisa registrar a ordem de chegada dos 04 caminhões no pátio de descarregamento. Para garantir que os dados não sejam perdidos caso o sistema reinicie, as informações devem ser salvas em um arquivo de texto e posteriormente recuperadas. Nosso objetivo é:

- acessar uma lista com 4 placas de veículos
- armazená-las em um arquivo
- acessar o arquivo e imprimir todo o seu conteúdo

Implementação: Parte 1

```
1 placas = ["ABC-1234", "XYZ-5678", "KGB-0007", "MNO-9988"]
2
3 with open("patio.txt", "w") as arquivo:
4     for placa in placas:
5         arquivo.write(placa + "\n")
6
7 print("Arquivo 'patio.txt' salvo com sucesso!")
```

Implementação: Parte 2

```
1 lista_recuperada = []
2
3 with open("patio.txt", "r") as arquivo:
4     for linha in arquivo:
5         linha_limpa = ""
6
7         for caractere in linha:
8             if caractere != "\n":
9                 linha_limpa += caractere
10
11         lista_recuperada.append(linha_limpa)
12
13 print(lista_recuperada)
```


Implementação alternativa: Parte 2

```
1 lista_recuperada = []  
2  
3 with open("patio.txt", "r") as arquivo:  
4     for linha in arquivo:  
5         lista_recuperada.append(linha.strip())  
6  
7 print(lista_recuperada)
```


Implementação alternativa: Parte 2

```
1 lista_recuperada = []
2
3 with open("patio.txt", "r") as arquivo:
4     for linha in arquivo:
5         lista_recuperada.append(linha.strip())
6
7 print(lista_recuperada)
```

A função strip() remove o '\n' do final de cada linha



Agora, é sua vez!

Você foi contratado para criar um sistema de controle de estoque de uma papelaria. O estoque deve ser guardado em um arquivo de texto chamado `estoque.txt`, onde cada linha contém o nome do produto e a quantidade separados por uma vírgula (exemplo: `Caneta,15`). Nosso objetivo é:

- Criar um dicionário com os produtos e estoques
- Armazená-los no arquivo, um por linha, separando chave de valor por vírgula
- Acessar o arquivo e imprimir todo o seu conteúdo

Implementação

```
1  dicionario_estoque = {  
2      "Caderno": 10,  
3      "Borracha": 25,  
4      "Lápis": 50  
5  }  
6  
7  nome_arquivo = "estoque.txt"  
8  
9  with open(nome_arquivo, "w", encoding="utf-8") as arquivo:  
10     for chave, valor in dicionario_estoque.items():  
11         arquivo.write(f"{chave},{valor}\n")
```



Agora, é sua vez!

Você foi contratado para criar um sistema de controle de estoque de uma papelaria. O estoque deve ser guardado em um arquivo de texto chamado `estoque.txt`, onde cada linha contém o nome do produto e a quantidade separados por uma vírgula (exemplo: `Caneta,15`). Nosso objetivo é:

- Criar um dicionário com os produtos e estoques
- Armazená-los em um arquivo JSON
- Recuperar do arquivo JSON e trabalhar novamente com um dicionário



Agora, é sua vez!

Dicionário Python:

```
1 dicionario_estoque = {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }
```

Formato JSON:

```
1 {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }
```

Implementação: Parte 1

```
1 ▾ dicionario_estoque = {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }  
6  
7 ▾ with open("estoque_manual.json", "w", encoding="utf-8") as arquivo:  
8     arquivo.write("{\n")  
9  
10     itens = list(dicionario_estoque.items())  
11     total_itens = len(itens)  
12  
13 ▾ for i in range(total_itens):  
14     chave, valor = itens[i]  
15  
16     linha_json = f'    "{chave}": {valor}'  
17  
18 ▾     if i < total_itens - 1:  
19         linha_json += ",\n"  
20 ▾     else:  
21         linha_json += "\n"  
22  
23     arquivo.write(linha_json)  
24  
25     arquivo.write("}\n")
```


Implementação: Parte 1

```
1 ▾ dicionario_estoque = {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }  
6  
7 ▾ with open("estoque_manual.json", "w", encoding="utf-8") as arquivo:  
8     arquivo.write("{\n")  
9  
10    itens = list(dicionario_estoque.items())  
11    total_itens = len(itens)  
12  
13 ▾ for i in range(total_itens):  
14     chave, valor = itens[i]  
15  
16     linha_json = f'    "{chave}": {valor}'  
17  
18 ▾     if i < total_itens - 1:  
19         linha_json += ",\n"  
20 ▾     else:  
21         linha_json += "\n"  
22  
23     arquivo.write(linha_json)  
24  
25    arquivo.write("}\n")
```

Converte um iterável em uma lista.
Neste caso, um dicionário.

Implementação: Parte 1

```
1 ▾ dicionario_estoque = {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }  
6  
7 ▾ with open("estoque_manual.json", "w", encoding="utf-8") as arquivo:  
8     arquivo.write("{\n")  
9  
10     itens = list(dicionario_estoque.items())  
11     total_itens = len(itens)  
12  
13 ▾ for i in range(total_itens):  
14     chave, valor = itens[i]  
15  
16     linha_json = f'    "{chave}": {valor}'  
17  
18 ▾     if i < total_itens - 1:  
19         linha_json += ",\n"  
20 ▾     else:  
21         linha_json += "\n"  
22  
23     arquivo.write(linha_json)  
24  
25     arquivo.write("}\n")
```

Cada item da lista é uma tupla, retorno da função .items()

Implementação: Parte 1

```
1 ▾ dicionario_estoque = {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }  
6  
7 ▾ with open("estoque_manual.json", "w", encoding="utf-8") as arquivo:  
8     arquivo.write("{\n")  
9  
10     itens = list(dicionario_estoque.items())  
11     total_itens = len(itens)  
12  
13 ▾ for i in range(total_itens):  
14     chave, valor = itens[i]  
15  
16     linha_json = f'    "{chave}": {valor}'  
17  
18 ▾     if i < total_itens - 1:  
19         linha_json += ",\n"  
20 ▾     else:  
21         linha_json += "\n"  
22  
23     arquivo.write(linha_json)  
24  
25     arquivo.write("}\n")
```

Formato de uma linha JSON.

Implementação: Parte 1

```
1 ▾ dicionario_estoque = {  
2     "Caderno": 10,  
3     "Borracha": 25,  
4     "Lápis": 50  
5 }  
6  
7 ▾ with open("estoque_manual.json", "w", encoding="utf-8") as arquivo:  
8     arquivo.write("{\n")  
9  
10     itens = list(dicionario_estoque.items())  
11     total_itens = len(itens)  
12  
13 ▾ for i in range(total_itens):  
14     chave, valor = itens[i]  
15  
16     linha_json = f'    "{chave}": {valor}'  
17  
18 ▾     if i < total_itens - 1:  
19         linha_json += ",\n"  
20 ▾     else:  
21         linha_json += "\n"  
22  
23     arquivo.write(linha_json)  
24  
25     arquivo.write("}\n")
```

Sem a vírgula na última linha.

Implementação: Parte 2

```
1 dicionario_recuperado = {}
2
3 with open("estoque_manual.json", "r", encoding="utf-8") as arquivo:
4     for linha in arquivo:
5         linha_limpa = linha.strip("\n\r\t,{}")
6
7         if not linha_limpa:
8             continue
9
10        parte_chave = ""
11        parte_valor = ""
12        passou_dos_dois_pontos = False
13
14        for caractere in linha_limpa:
15            if caractere == ":" and not passou_dos_dois_pontos:
16                passou_dos_dois_pontos = True
17                continue
18
19            if not passou_dos_dois_pontos:
20                parte_chave += caractere
21            else:
22                parte_valor += caractere
23
24        chave_final = parte_chave.strip(' ' )
25        valor_final = int(parte_valor.strip())
26
27        dicionario_recuperado[chave_final] = valor_final
28
29 print(dicionario_recuperado)
```

A função strip() limpa a linha sob o ponto de vista do que for passado por parâmetro.

Implementação: Parte 2

```
1 dicionario_recuperado = {}
2
3 with open("estoque_manual.json", "r", encoding="utf-8") as arquivo:
4     for linha in arquivo:
5         linha_limpa = linha.strip(" \n\r\t,{}")
6
7         if not linha_limpa:
8             continue
9
10        parte_chave = ""
11        parte_valor = ""
12        passou_dos_dois_pontos = False
13
14        for caractere in linha_limpa:
15            if caractere == ":" and not passou_dos_dois_pontos:
16                passou_dos_dois_pontos = True
17                continue
18
19            if not passou_dos_dois_pontos:
20                parte_chave += caractere
21            else:
22                parte_valor += caractere
23
24        chave_final = parte_chave.strip(' ' ' ')
25        valor_final = int(parte_valor.strip())
26
27        dicionario_recuperado[chave_final] = valor_final
28
29 print(dicionario_recuperado)
```

Se uma linha ficar vazia, ou seja for "", é avaliada como *False*.

Implementação: Parte 2

```
1 dicionario_recuperado = {}
2
3 with open("estoque_manual.json", "r", encoding="utf-8") as arquivo:
4     for linha in arquivo:
5         linha_limpa = linha.strip("\n\r\t,{}")
6
7         if not linha_limpa:
8             continue
9
10        parte_chave = ""
11        parte_valor = ""
12        passou_dos_dois_pontos = False
13
14        for caractere in linha_limpa:
15            if caractere == ":" and not passou_dos_dois_pontos:
16                passou_dos_dois_pontos = True
17                continue
18
19            if not passou_dos_dois_pontos:
20                parte_chave += caractere
21            else:
22                parte_valor += caractere
23
24        chave_final = parte_chave.strip('\"')
25        valor_final = int(parte_valor.strip())
26
27        dicionario_recuperado[chave_final] = valor_final
28
29 print(dicionario_recuperado)
```

Semelhante ao *break*, interrompe a iteração atual e avança para a próxima. o *break* interrompe o loop.

Implementação: Parte 2

```
1 dicionario_recuperado = {}
2
3 with open("estoque_manual.json", "r", encoding="utf-8") as arquivo:
4     for linha in arquivo:
5         linha_limpa = linha.strip("\n\r\t,{}")
6
7         if not linha_limpa:
8             continue
9
10        parte_chave = ""
11        parte_valor = ""
12        passou_dos_dois_pontos = False
13
14        for caractere in linha_limpa:
15            if caractere == ":" and not passou_dos_dois_pontos:
16                passou_dos_dois_pontos = True
17                continue
18
19            if not passou_dos_dois_pontos:
20                parte_chave += caractere
21            else:
22                parte_valor += caractere
23
24        chave_final = parte_chave.strip(' ')
25        valor_final = int(parte_valor.strip())
26
27        dicionario_recuperado[chave_final] = valor_final
28
29 print(dicionario_recuperado)
```

Implementação manual da função split(":")

Implementação: Parte 2

```
1 dicionario_recuperado = {}
2
3 with open("estoque_manual.json", "r", encoding="utf-8") as arquivo:
4     for linha in arquivo:
5         linha_limpa = linha.strip("\n\r\t,{}")
6
7         if not linha_limpa:
8             continue
9
10        parte_chave = ""
11        parte_valor = ""
12        passou_dos_dois_pontos = False
13
14        for caractere in linha_limpa:
15            if caractere == ":" and not passou_dos_dois_pontos:
16                passou_dos_dois_pontos = True
17                continue
18
19            if not passou_dos_dois_pontos:
20                parte_chave += caractere
21            else:
22                parte_valor += caractere
23
24        chave_final = parte_chave.strip(' ')
25        valor_final = int(parte_valor.strip())
26
27        dicionario_recuperado[chave_final] = valor_final
28
29 print(dicionario_recuperado)
```

`partes = linha_limpa.split(":")`

A função `split(":")` retorna uma lista com 02 elementos.



PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.